A trace semantics for the process calculus of ProVerif

1 The trace semantics

1.1 Messages

Definition 1.1 (Signature, terms)**.** A *signature* is a family $\Sigma = (\Sigma_n)_{n\in\mathbb{N}}$ of sets; $f \in \Sigma_n$ is a constructor of arity n. The set $\mathcal{T} = \mathcal{T}(\Sigma)$ of *raw terms* is generated by

$$t ::= \nu_k \ (k \in \mathbb{N}) \ \mid \ f(t_1, \dots, t_n) \ (f \in \Sigma_n).$$

A *substitution* is a map $\sigma : \mathbb{N} \to \mathcal{T}$; $t\sigma$ replaces each ν_k in t by $\sigma(k)$.

Definition 1.2 (Theory)**.** A *theory* over Σ is a pair (E, Fr) with $E \subseteq \mathcal{T} \times \mathcal{T}$ and $\mathrm{Fr} \subseteq \bigcup_n \Sigma_n$ such that

(H) for every $(l, r) \in E$, both l and r are of the form $f(\dots)$ with $f \notin \mathrm{Fr}$.

The relation $\equiv_E$ is the least congruence on $\mathcal{T}$ containing $l\sigma \equiv_E r\sigma$ for all $(l, r) \in E$ and all substitutions σ. The set of *messages* is $\mathcal{M} = \mathcal{T}/\!\equiv_E$, with $[t]$ the class of t. Constructors act on $\mathcal{M}$ by $f([t_1], \dots, [t_n]) = [f(t_1, \dots, t_n)]$, and ν_k also denotes $[\nu_k]$.

A Σ-algebra A *satisfies* E if $l\sigma_A = r\sigma_A$ for every $(l, r) \in E$ and every valuation $\sigma_A : \mathbb{N} \to A$, where $t\sigma_A$ is the evaluation of t in A.

Proposition 1.3. *$\mathcal{M}$ satisfies E. For every Σ-algebra A satisfying E and every $v : \mathbb{N} \to A$ there is a unique homomorphism $\mathcal{M} \to A$ sending ν_k to $v(k)$.*

Definition 1.4 (View)**.** Let $\mathcal{V} = \mathbb{N} \uplus \{f(m_1, \dots, m_n) \mid f \in \mathrm{Fr} \cap \Sigma_n,\ m_i \in \mathcal{M}\} \uplus \{\bot\}$. Define $\mathrm{view}_0 : \mathcal{T} \to \mathcal{V}$ by

$$\mathrm{view}_0(\nu_k) = k, \qquad \mathrm{view}_0(f(t_1, \dots, t_n)) = \begin{cases} f([t_1], \dots, [t_n]) & f \in \mathrm{Fr}, \\ \bot & f \notin \mathrm{Fr}. \end{cases}$$

Proposition 1.5. *The following hold.*

1. *$s \equiv_E t$ implies $\mathrm{view}_0(s) = \mathrm{view}_0(t)$; hence view_0 induces $\mathrm{view} : \mathcal{M} \to \mathcal{V}$.*
2. *$\mathrm{view}(f(m_1, \dots, m_n)) = f(m_1, \dots, m_n)$ if $f \in \mathrm{Fr}$, and $\bot$ otherwise.*
3. *If $\mathrm{view}(m) = f(m_1, \dots, m_n)$ then $m = f(m_1, \dots, m_n)$.*
4. *$k \mapsto \nu_k$ is injective $\mathbb{N} \to \mathcal{M}$.*

Definition 1.6 (May-fail messages, lifting)**.** Let $\mathcal{M}_\perp = \mathcal{M} \uplus \{\perp\}$. For $g : \mathcal{M}^n \to \mathcal{M}_\perp$ let $g^\uparrow : \mathcal{M}_\perp^n \to \mathcal{M}_\perp$ be $g^\uparrow(\vec{u}) = g(\vec{u})$ if every $u_i \in \mathcal{M}$ and $\perp$ otherwise; $f^\uparrow$ for $f \in \Sigma_n$ is the lift of $f : \mathcal{M}^n \to \mathcal{M} \subseteq \mathcal{M}_\perp$. A *destructor* of arity n is a function $\mathcal{M}_\perp^n \to \mathcal{M}_\perp$.

Definition 1.7 (Destructors from rewrite rules)**.** Fix disjoint countable sets of *ordinary variables* x and *may-fail variables* u. Patterns over Fr are generated by $p ::= x \mid u \mid f(p_1, \ldots, p_n)$ with $f \in \mathrm{Fr} \cap \Sigma_n$. A *matching* of $\vec{u} \in \mathcal{M}_\perp^n$ against $\vec{p}$ is a map θ from the variables of $\vec{p}$ to $\mathcal{M}_\perp$, sending ordinary variables into $\mathcal{M}$, such that $p_i\theta = u_i$ for all i, where $f(p_1, \ldots, p_n)\theta = f(p_1\theta, \ldots, p_n\theta)$ if every $p_j\theta \in \mathcal{M}$ and is undefined otherwise. By Proposition 1.5, at most one matching exists.

A *right-hand side* is $q ::= \mathsf{fail} \mid u \mid M$, where M is a term over constructors and ordinary variables; $q\theta \in \mathcal{M}_\perp$ is $\perp$, $\theta(u)$, and the evaluation of M in $\mathcal{M}$ under θ respectively. A list of rules $g(\vec{p}_1) \to q_1;\ \ldots;\ g(\vec{p}_k) \to q_k$, each variable of q_i occurring in $\vec{p}_i$, defines the destructor

$$g(\vec{u}) = \begin{cases} q_j\theta & j \text{ least such that a matching } \theta \text{ of } \vec{u} \text{ against } \vec{p}_j \text{ exists,} \\ \perp & \text{if there is none.} \end{cases}$$

Definition 1.8 (Public operations, deduction)**.** Let $\mathrm{Pub} \subseteq \bigcup_n \Sigma_n$ and let Dt be a set of destructors. Set $\mathrm{Ops} = \{f^\uparrow \mid f \in \mathrm{Pub}\} \cup \mathrm{Dt}$. A set $S \subseteq \mathcal{M}$ is Ops-*closed* if for every $o \in \mathrm{Ops}$ of arity n and every $\vec{u} \in (S \uplus \{\perp\})^n$, $o(\vec{u}) \in \mathcal{M}$ implies $o(\vec{u}) \in S$. For $K \subseteq \mathcal{M}$,

$$\langle K \rangle = \bigcap \{S \supseteq K \mid S \text{ is Ops-closed}\}, \qquad K \vdash m \iff m \in \langle K \rangle.$$

Proposition 1.9. *$\langle\cdot\rangle$ is a closure operator on $\mathcal{P}(\mathcal{M})$, $\langle K \rangle$ is* Ops*-closed, and $K \vdash m_1, \ldots, K \vdash m_n$ imply $K \vdash f(m_1, \ldots, m_n)$ for $f \in \mathrm{Pub} \cap \Sigma_n$. In particular $\emptyset \vdash a$ for $a \in \mathrm{Pub} \cap \Sigma_0$.*

1.2 Traces and processes

Definition 1.10 (Actions, traces)**.**

$$\mathrm{Act} = \{\mathsf{out}(c, m),\ \mathsf{in}(c, m),\ \mathsf{new}(k),\ \mathsf{ev}(m),\ \mathsf{say}(m) \mid c, m \in \mathcal{M},\ k \in \mathbb{N}\}.$$

A *trace* is a word $t \in \mathrm{Act}^*$; ε is the empty trace and $l \le t$ means l is a prefix of t. The actions $\mathsf{out}(c, m)$ and $\mathsf{in}(c, m)$ are *unfinished*; $\mathrm{Closed}(t)$ holds if no action of t is unfinished. $\mathrm{nonces}(t) \in \mathbb{N}^*$ is the image of t under the monoid homomorphism with $\mathsf{new}(k) \mapsto k$ and every other action $\mapsto \varepsilon$. A *process* is a subset of Act^*; $\mathrm{Proc} = \mathcal{P}(\mathrm{Act}^*)$.

Definition 1.11 (Sequential constructors)**.** For $a \in \mathrm{Act}$, $P \in \mathrm{Proc}$, $F : \mathcal{M} \to \mathrm{Proc}$:

$$\begin{aligned} a \cdot P &= \{\varepsilon\} \cup \{a\,t \mid t \in P\}, & \mathbf{0} &= \{\varepsilon\}, \\ \mathsf{out}(c, m).P &= \mathsf{out}(c, m) \cdot P, & \mathsf{ev}(m).P &= \mathsf{ev}(m) \cdot P, \\ \mathsf{in}(c).F &= \textstyle\bigcup_{m \in \mathcal{M}} \mathsf{in}(c, m) \cdot F(m), & \mathsf{new}.F &= \textstyle\bigcup_{k \in \mathbb{N}} \mathsf{new}(k) \cdot \{t \in F(\nu_k) \mid k \notin \mathrm{nonces}(t)\}. \end{aligned}$$

Definition 1.12 (Parallel composition)**.** Let I be a set. For $s \in (I \times \mathrm{Act})^*$ and $i \in I$, $s\restriction i \in \mathrm{Act}^*$ is the word of actions of s tagged i, in order, and $\overline{s} \in \mathrm{Act}^*$ is s with tags removed. For $f : I \to \mathrm{Act}^*$, $\mathrm{Interleaves}(f, s)$ holds if $s\restriction i = f(i)$ for all i. Define $\mathrm{comms} : (I \times \mathrm{Act})^* \to \mathcal{P}(\mathrm{Act}^*)$ by recursion on the word:

$$\begin{aligned} \mathrm{comms}(\varepsilon) &= \{\varepsilon\}, \\ \mathrm{comms}\big((i, \mathsf{out}(c, m))\,(j, \mathsf{in}(c', m'))\,s\big) &= \mathsf{out}(c, m) \cdot \mathrm{comms}\big((j, \mathsf{in}(c', m'))\,s\big) \\ &\quad \cup\ \{t \in \mathrm{comms}(s) \mid i \neq j,\ c = c',\ m = m'\}, \\ \mathrm{comms}((i, a)\,s) &= a \cdot \mathrm{comms}(s) \quad \text{in all other cases,} \end{aligned}$$

where $a \cdot X = \{a\,t \mid t \in X\}$. For $P : I \to \mathrm{Proc}$,

$$\mathrm{merge}_I(P) = \{t \mid \exists f, s.\ (\forall i.\ f(i) \in P(i)) \ \wedge\ \mathrm{Interleaves}(f, s) \\ \wedge\ t \in \mathrm{comms}(s) \ \wedge\ \mathrm{nonces}(t) \text{ has no repeated letter}\}.$$

$P \mid Q = \mathrm{merge}_{\{0,1\}}(0 \mapsto P,\ 1 \mapsto Q)$ and $!P = \mathrm{merge}_{\mathbb{N}}(k \mapsto P)$.

Proposition 1.13. *Let $t \in \mathrm{comms}(s)$.*

1. $\mathrm{nonces}(t) = \mathrm{nonces}(\overline{s})$.
2. *If a occurs in t then (i, a) occurs in s for some i.*
3. *If $\mathrm{Closed}(t)$ and $(k, \mathsf{in}(c, m))$ occurs in s, then $(i, \mathsf{out}(c, m))$ occurs in s for some $i \neq k$.*
4. *$!P = P \mid !P$ for every $P \in \mathrm{Proc}$.*

1.3 Translation of processes

Definition 1.14 (Source syntax)**.** Fix a set of variables $x, y, \ldots$ and a set of names $a, b, c, \ldots$ containing the free names $\mathcal{N}_{\mathrm{pub}} \uplus \mathcal{N}_{\mathrm{priv}}$. Terms, expressions and processes are generated by

$$\begin{aligned} M, N &::= x \mid a \mid f(M_1, \ldots, M_n), \\ D &::= M \mid h(D_1, \ldots, D_n) \mid \mathsf{fail}, \\ P, Q &::= \mathbf{0} \mid \mathsf{out}(N, M); P \mid \mathsf{in}(N, x); P \mid P \mid Q \mid\, !P \mid \mathsf{new}\ a; P \\ &\quad \mid \mathsf{let}\ x = D\ \mathsf{in}\ P\ \mathsf{else}\ Q \mid \mathsf{if}\ M\ \mathsf{then}\ P\ \mathsf{else}\ Q \mid \mathsf{event}(M); P, \end{aligned}$$

where f ranges over constructors and h over constructors and destructors. Types are erased.

Definition 1.15 (Model)**.** A *model* consists of a signature Σ whose nullary symbols include $\mathcal{N}_{\mathrm{pub}} \uplus \mathcal{N}_{\mathrm{priv}}$, a theory (E, Fr) over Σ, a finite set of destructors each given by rules as in Definition 1.7, sets Pub, Dt of public constructors and public destructors, and a closed process P_0 with $\mathrm{fn}(P_0) \subseteq \mathcal{N}_{\mathrm{pub}} \uplus \mathcal{N}_{\mathrm{priv}}$.

Definition 1.16 (Translation)**.** An *environment* ρ maps variables and names to $\mathcal{M}$; ρ_0 maps each free name a to the constant $a \in \mathcal{M}$. Write $\rho M \in \mathcal{M}$ for the homomorphic extension. Expressions are evaluated in $\mathcal{M}_\perp$:

$$[\![M]\!]\rho = \rho M, \qquad [\![\mathsf{fail}]\!]\rho = \perp, \qquad [\![f(\vec{D})]\!]\rho = f^{\uparrow}([\![\vec{D}]\!]\rho), \qquad [\![g(\vec{D})]\!]\rho = g([\![\vec{D}]\!]\rho)$$

for a constructor f and a destructor g. Processes are translated by

$$\begin{aligned} [\![\mathbf{0}]\!]\rho &= \mathbf{0}, & [\![\mathsf{out}(N, M); P]\!]\rho &= \mathsf{out}(\rho N, \rho M).[\![P]\!]\rho, \\ [\![\mathsf{event}(M); P]\!]\rho &= \mathsf{ev}(\rho M).[\![P]\!]\rho, & [\![\mathsf{in}(N, x); P]\!]\rho &= \mathsf{in}(\rho N).\big(m \mapsto [\![P]\!]\rho[x \mapsto m]\big), \\ [\![P \mid Q]\!]\rho &= [\![P]\!]\rho \mid [\![Q]\!]\rho, & [\![\mathsf{new}\ a; P]\!]\rho &= \mathsf{new}.\big(m \mapsto [\![P]\!]\rho[a \mapsto m]\big), \end{aligned}$$

$$[\![!P]\!]\rho = !\,[\![P]\!]\rho, \qquad [\![\mathsf{let}\ x = D\ \mathsf{in}\ P\ \mathsf{else}\ Q]\!]\rho = \begin{cases} [\![P]\!]\rho[x \mapsto m] & [\![D]\!]\rho = m \in \mathcal{M}, \\ [\![Q]\!]\rho & [\![D]\!]\rho = \perp, \end{cases}$$

$$[\![\mathsf{if}\ M\ \mathsf{then}\ P\ \mathsf{else}\ Q]\!]\rho = \begin{cases} [\![P]\!]\rho & \rho M = \mathsf{true}, \\ [\![Q]\!]\rho & \rho M \neq \mathsf{true}. \end{cases}$$

The translation of P_0 is $[\![P_0]\!]\rho_0$. A translated process contains no say action and mentions no ν_k except through new.

Remark 1.17. In the development a process is a program in the continuation monad with answer type Proc, run by $\mathrm{run}(b) = b(\lambda_.\,\mathbf{0})$, with primitives $\mathrm{send}(c,m) = \lambda\kappa.\,\mathsf{out}(c,m).\kappa()$, $\mathrm{recv}(c) = \lambda\kappa.\,\mathsf{in}(c).\kappa$, $\mathrm{fresh} = \lambda\kappa.\,\mathsf{new}.\kappa$, $\mathrm{emit}(m) = \lambda\kappa.\,\mathsf{ev}(m).\kappa()$, $\mathrm{fork} = \lambda\kappa.\,\kappa(\mathrm{tt}) \mid \kappa(\mathrm{ff})$, $\mathrm{repl} = \lambda\kappa.\,!\kappa()$ and $\mathrm{stop} = \lambda_.\,\mathbf{0}$; let and if are case analysis in the metalanguage. The clauses above are the unfolding of that presentation.

1.4 The attacker and the security properties

Definition 1.18 (Enemy)**.** For $K \subseteq \mathcal{M}$, $\mathrm{Enemy}(K) \subseteq \mathrm{Act}^*$ is the least family of sets such that

1. $\varepsilon \in \mathrm{Enemy}(K)$;
2. if $K \vdash c$ and $e \in \mathrm{Enemy}(K \cup \{m\})$ then $\mathsf{in}(c,m)\,e \in \mathrm{Enemy}(K)$, for every $m \in \mathcal{M}$;
3. if $K \vdash c$, $K \vdash m$ and $e \in \mathrm{Enemy}(K)$ then $\mathsf{out}(c,m)\,e \in \mathrm{Enemy}(K)$;
4. if $k \notin \mathrm{nonces}(e)$ and $e \in \mathrm{Enemy}(K \cup \{\nu_k\})$ then $\mathsf{new}(k)\,e \in \mathrm{Enemy}(K)$;
5. if $K \vdash m$ and $e \in \mathrm{Enemy}(K)$ then $\mathsf{say}(m)\,e \in \mathrm{Enemy}(K)$.

Definition 1.19 (Attacks, secrecy, correspondence)**.** For $P \in \mathrm{Proc}$,

$$\mathrm{Attacks}(P) = \{t \in P \mid \mathrm{Enemy}(\emptyset) \ :\ \mathrm{Closed}(t)\}.$$

For $s \in \mathcal{M}$,

$$\mathrm{Secret}(P,s) \iff \forall t \in \mathrm{Attacks}(P).\ \mathsf{say}(s) \notin t.$$

For a set V and relations $\mathrm{Hyp}, \mathrm{Concl} \subseteq V \times \mathrm{Act}$,

$$\mathrm{Corr}(P,\mathrm{Hyp},\mathrm{Concl}) \iff \forall t \in \mathrm{Attacks}(P).\ \forall l \le t.\ \forall a.\ \big(l = l'a \ \wedge\ \exists\sigma.\,\mathrm{Hyp}(\sigma,a)\big) \Rightarrow \exists\sigma.\,\mathrm{Hyp}(\sigma,a) \ \wedge\ \exists b \in l.\,\mathrm{Concl}(\sigma,b).$$

Proposition 1.20 (Prefix-closure)**.** $\mathbf{0}$ *and* $\mathrm{Enemy}(K)$ *are prefix-closed. If* P*, every* $F(m)$ *and every* $P(i)$ *are prefix-closed, so are* $a \cdot P$*,* $\mathsf{in}(c).F$*,* $\mathsf{new}.F$ *and* $\mathrm{merge}_I(P)$*. If* P *is prefix-closed, so is* $\mathrm{Attacks}(P)$*.*

Definition 1.21 (Queries)**.** A *fact* is $\mathsf{event}(M)$ or $\mathsf{attacker}(M)$ with M a term over constructors, free names and a finite set X of variables; it is an *event fact* or an *attacker fact* accordingly. For a valuation $\sigma : X \to \mathcal{M}$ write $\sigma M \in \mathcal{M}$ for the evaluation of M with free names as constants. For a fact F let

$$\mathrm{Hyp}_F(\sigma,a) \iff \begin{cases} a = \mathsf{ev}(\sigma M) & F = \mathsf{event}(M), \\ a = \mathsf{say}(\sigma M) & F = \mathsf{attacker}(M). \end{cases}$$

The query $\mathsf{query}\ \mathsf{attacker}(M)$, for M without variables, denotes $\mathrm{Secret}([\![P_0]\!]\rho_0, \rho_0 M)$. The query $\mathsf{query}\ F \Longrightarrow H_1 \vee \cdots \vee H_m$ with facts $F, H_1, \ldots, H_m$ over X denotes $\mathrm{Corr}([\![P_0]\!]\rho_0, \mathrm{Hyp}, \mathrm{Concl})$ with $V = \{1,\ldots,m\} \times (X \to \mathcal{M})$, $\mathrm{Hyp}((j,\sigma),a) \iff \mathrm{Hyp}_F(\sigma,a)$ and $\mathrm{Concl}((j,\sigma),b) \iff \mathrm{Hyp}_{H_j}(\sigma,b)$.

Lemma 1.22. *Let* $e \in \mathrm{Enemy}(K)$ *and let* $S \subseteq \mathcal{M}$ *be* Ops*-closed with* $K \subseteq S$*,* $\nu_k \in S$ *for all* k*, and* $m \in S$ *whenever* $\mathsf{in}(c,m)$ *occurs in* e*. Then* $m \in S$ *whenever* $\mathsf{say}(m)$ *occurs in* e*.*

Theorem 1.23 (Closed-set method)**.** *Let* $P \in \mathrm{Proc}$ *and let* $S \subseteq \mathcal{M}$ *be* Ops*-closed with* $\nu_k \in S$ *for all* k*. Suppose that for every* $t \in P$*,* $\mathsf{out}(c,m) \in t$ *implies* $m \in S$*, and no* $t \in P$ *contains a* say *action. Then* $s \notin S$ *implies* $\mathrm{Secret}(P,s)$*.*

2 Relation to the operational semantics of ProVerif

2.1 The reduction semantics

This section restates the semantics of Blanchet's survey (§2.4, §2.5.1, §3.1–3.3) for the syntax of Definition 1.14. Names are atoms; substitutions map variables to terms and never substitute fail or a may-fail variable for an ordinary variable. An equational theory E is a finite set of equations between terms without names; $=_E$ is the closure of E under reflexivity, symmetry, transitivity, application of function symbols and substitution. Each constructor f carries the rule $f(\vec{x}) \to f(\vec{x})$, each destructor g a finite ordered list of rules $g(\vec{U}) \to U$ between may-fail terms without names, and every $\mathrm{def}(h)$ ends with $h(\vec{u}) \to \mathsf{fail}$ for may-fail variables $\vec{u}$.

Evaluation $D \Downarrow U$ of closed expressions to closed may-fail terms is the least relation with $M \Downarrow M$, $\mathsf{fail} \Downarrow \mathsf{fail}$, and $h(D_1, \ldots, D_n) \Downarrow \sigma U'_j$ whenever $D_i \Downarrow U_i$ for all i, $\mathrm{def}(h) = (h(U'_{i,1}, \ldots, U'_{i,n}) \to U'_i)_{i \le k}$, $\sigma U'_{j,i} =_E U_i$ for all i, and for all $i < j$ and all σ' there is some l with $\sigma' U'_{i,l} \neq_E U_l$.

A *configuration* is $(\mathcal{N}_{\mathrm{pub}}, \mathcal{N}_{\mathrm{priv}}), \mathcal{P}$ with $\mathcal{N}_{\mathrm{pub}}, \mathcal{N}_{\mathrm{priv}}$ finite disjoint sets of names and $\mathcal{P}$ a finite multiset of closed processes with $\mathrm{fn}(\mathcal{P}) \subseteq \mathcal{N}_{\mathrm{pub}} \cup \mathcal{N}_{\mathrm{priv}}$. Writing $\mathcal{E}$ for the pair of name sets, reduction is the least relation closed under

$$\mathcal{E}, \mathcal{P} \cup \{\mathbf{0}\} \to \mathcal{E}, \mathcal{P} \qquad \text{(Red Nil)}$$

$$\mathcal{E}, \mathcal{P} \cup \{P \mid Q\} \to \mathcal{E}, \mathcal{P} \cup \{P, Q\} \qquad \text{(Red Par)}$$

$$\mathcal{E}, \mathcal{P} \cup \{!P\} \to \mathcal{E}, \mathcal{P} \cup \{P, !P\} \qquad \text{(Red Repl)}$$

$$(\mathcal{N}_{\mathrm{pub}}, \mathcal{N}_{\mathrm{priv}}), \mathcal{P} \cup \{\mathsf{new}\ a; P\} \to (\mathcal{N}_{\mathrm{pub}}, \mathcal{N}_{\mathrm{priv}} \cup \{a'\}), \mathcal{P} \cup \{P\{a'/a\}\} \quad \text{if } a' \notin \mathcal{N}_{\mathrm{pub}} \cup \mathcal{N}_{\mathrm{priv}} \qquad \text{(Red Res)}$$

$$\mathcal{E}, \mathcal{P} \cup \{\mathsf{out}(N, M); Q,\ \mathsf{in}(N', x); P\} \to \mathcal{E}, \mathcal{P} \cup \{Q,\ P\{M/x\}\} \quad \text{if } N =_E N' \qquad \text{(Red I/O}')$$

$$\mathcal{E}, \mathcal{P} \cup \{\mathsf{let}\ x = D\ \mathsf{in}\ P\ \mathsf{else}\ Q\} \to \mathcal{E}, \mathcal{P} \cup \{P\{M/x\}\} \quad \text{if } D \Downarrow M \qquad \text{(Red Eval 1)}$$

$$\mathcal{E}, \mathcal{P} \cup \{\mathsf{let}\ x = D\ \mathsf{in}\ P\ \mathsf{else}\ Q\} \to \mathcal{E}, \mathcal{P} \cup \{Q\} \quad \text{if } D \Downarrow \mathsf{fail} \qquad \text{(Red Eval 2)}$$

$$\mathcal{E}, \mathcal{P} \cup \{\mathsf{if}\ M\ \mathsf{then}\ P\ \mathsf{else}\ Q\} \to \mathcal{E}, \mathcal{P} \cup \{P\} \quad \text{if } M =_E \mathsf{true} \qquad \text{(Red Cond 1}')$$

$$\mathcal{E}, \mathcal{P} \cup \{\mathsf{if}\ M\ \mathsf{then}\ P\ \mathsf{else}\ Q\} \to \mathcal{E}, \mathcal{P} \cup \{Q\} \quad \text{if } M \neq_E \mathsf{true} \qquad \text{(Red Cond 2}')$$

$$\mathcal{E}, \mathcal{P} \cup \{\mathsf{event}(M); P\} \to \mathcal{E}, \mathcal{P} \cup \{P\} \qquad \text{(Red Event)}$$

A *trace* is a finite reduction sequence $\mathcal{E}_0, \mathcal{P}_0 \to \cdots \to \mathcal{E}_n, \mathcal{P}_n$, written $\mathcal{E}_0, \mathcal{P}_0 \to^* \mathcal{E}_n, \mathcal{P}_n$.

Definition 2.1 (Survey, Definitions 3.1–3.5)**.** Let P_0 be a closed process.

1. A closed process Q is an $\mathcal{N}_{\mathrm{pub}}$-*adversary* if $\mathrm{fn}(Q) \subseteq \mathcal{N}_{\mathrm{pub}}$, every function symbol of Q is public, and Q contains no event.

2. A trace *outputs M publicly* if it contains a step (Red I/O$'$) whose output is $\mathsf{out}(N, M); Q$ with $N \in \mathcal{N}_{\mathrm{pub}}$. It *executes* $\mathsf{event}(M)$ if it contains a step (Red Event) on $\mathsf{event}(M); P$.

3. P_0 *preserves the secrecy of* the closed term M *from* $\mathcal{N}_{\mathrm{pub}}$ if, for some $\mathcal{N}_{\mathrm{priv}}$ disjoint from $\mathcal{N}_{\mathrm{pub}}$ with $\mathrm{fn}(M) \cup \mathrm{fn}(P_0) \subseteq \mathcal{N}_{\mathrm{pub}} \cup \mathcal{N}_{\mathrm{priv}}$, no trace $(\mathcal{N}_{\mathrm{pub}}, \mathcal{N}_{\mathrm{priv}}), \{P_0, Q\} \to^* \mathcal{E}', \mathcal{P}'$ with Q an $\mathcal{N}_{\mathrm{pub}}$-adversary outputs M publicly.

4. P_0 *satisfies* $\mathsf{event}(M) \rightsquigarrow \bigvee_{j=1}^{m} \bigwedge_{k=1}^{l_j} \mathsf{event}(M_{jk})$ *against* $\mathcal{N}_{\mathrm{pub}}$*-adversaries* if, for some $\mathcal{N}_{\mathrm{priv}}$ as above, for every $\mathcal{N}_{\mathrm{pub}}$-adversary Q, every trace Tr from $(\mathcal{N}_{\mathrm{pub}}, \mathcal{N}_{\mathrm{priv}}), \{P_0, Q\}$ and every substitution σ: if Tr executes $\mathsf{event}(\sigma M)$ then there are σ' and j with $\sigma' M = \sigma M$ such that Tr executes $\mathsf{event}(\sigma' M_{jk})$ for every $k \le l_j$.

The Horn clauses for the adversary (survey, §3.2.2) are

$$\mathsf{attacker}(a[\,]) \text{ for } a \in \mathcal{N}_{\mathrm{pub}}, \tag{Init}$$

$$\mathsf{attacker}(b_0[x]), \tag{Rn}$$

$$\mathsf{attacker}(\mathsf{fail}), \tag{Rfail}$$

$$\mathsf{attacker}(U_{i,1}) \wedge \cdots \wedge \mathsf{attacker}(U_{i,n}) \wedge \textstyle\bigwedge_{j<i} \forall \tilde{v}_j.\, (U_{i,1}, \ldots, U_{i,n}) \neq (U_{j,1}, \ldots, U_{j,n}) \\ \Rightarrow \mathsf{attacker}(U_i), \tag{Rh}$$

$$\mathsf{message}(x, y) \wedge \mathsf{attacker}(x) \Rightarrow \mathsf{attacker}(y), \tag{Rl}$$

$$\mathsf{attacker}(x) \wedge \mathsf{attacker}(y) \Rightarrow \mathsf{message}(x, y), \tag{Rs}$$

one instance of (Rh) for each public h and each rule $h(U_{i,1}, \ldots, U_{i,n}) \to U_i$ of $\mathrm{def}(h)$.

Remark 2.2. Under Definition 1.7, $\langle\cdot\rangle$ is the least fixed point of (Init), (Rfail) and (Rh); clauses (Rl) and (Rs) are the in and out clauses of Enemy, and (Rn) is its new clause.

Fix $\mathcal{N}_{\mathrm{priv}}$ as in Definition 2.1, and for an $\mathcal{N}_{\mathrm{pub}}$-adversary Q write $\mathcal{C}_Q$ for the configuration $(\mathcal{N}_{\mathrm{pub}}, \mathcal{N}_{\mathrm{priv}}), \{P_0, Q\}$.

Definition 2.3 (Adversary knowledge)**.** In a trace Tr from $\mathcal{C}_Q$, tag P_0 as honest and Q as adversarial, each process introduced by a step inheriting the tag of the process whose continuation it is. $K(\mathrm{Tr})$ is the least set of closed terms that contains $\mathcal{N}_{\mathrm{pub}}$, every name created by a step (Red Res) on an adversarial process, every term M of a step (Red I/O′) whose input is adversarial, and every M with $h(D_1, \ldots, D_n) \Downarrow M$ for a public h and $D_1, \ldots, D_n \in K(\mathrm{Tr}) \cup \{\mathsf{fail}\}$. The trace Tr *satisfies* $\mathsf{attacker}(M)$ if $M =_E M'$ for some $M' \in K(\mathrm{Tr})$, and *satisfies* $\mathsf{event}(M)$ if it executes $\mathsf{event}(M)$.

Lemma 2.4. *Let $\mathcal{N}_{\mathrm{pub}} \neq \emptyset$ and let M be a closed term with $\mathrm{fn}(M) \subseteq \mathcal{N}_{\mathrm{pub}} \cup \mathcal{N}_{\mathrm{priv}}$. Then P_0 preserves the secrecy of M from $\mathcal{N}_{\mathrm{pub}}$ if and only if no trace from $\mathcal{C}_Q$, for any $\mathcal{N}_{\mathrm{pub}}$-adversary Q, satisfies $\mathsf{attacker}(M)$.*

Definition 2.5 (Correspondences between facts)**.** Let F and H_{jk} ($j \leq m$, $k \leq l_j$) be facts over X. P_0 *satisfies* $F \rightsquigarrow \bigvee_{j=1}^{m} \bigwedge_{k=1}^{l_j} H_{jk}$ *against $\mathcal{N}_{\mathrm{pub}}$-adversaries* if for every $\mathcal{N}_{\mathrm{pub}}$-adversary Q, every trace Tr from $\mathcal{C}_Q$ and every substitution σ: if Tr satisfies σF then there are σ' and j with $\sigma' F = \sigma F$ such that Tr satisfies $\sigma' H_{jk}$ for every $k \leq l_j$. For event facts this is Definition 2.1(4).

2.2 The correspondence

Throughout, fix a model (Definition 1.15) satisfying

(R1) Fr is the set of symbols at the root of no equation and (E, Fr) satisfies (H);

(R2) every rule of every destructor has patterns over Fr, and the destructor is the one of Definition 1.7;

(R3) Pub consists of the public constructors and the names in $\mathcal{N}_{\mathrm{pub}}$, and Dt of the public destructors;

(R4) P_0 is a closed process of Definition 1.14.

Names of the reduction semantics are the free names together with the created names; terms are interpreted in $\mathcal{M}$ through an injection ι from created names to $\mathbb{N}$, extended by $\iota a = a$ for free names, $\iota a' = \nu_{\iota a'}$ for created names, $\iota f(\vec{M}) = f(\iota\vec{M})$ and $\iota\,\mathsf{fail} = \bot$; ιM depends only on the class of M modulo E. Let ρ_ι be the environment with $\rho_\iota(a) = \iota a$ for every free or created name a.

Lemma 2.6 (Evaluation). *For every closed expression D over free and created names, some U satisfies $D \Downarrow U$, and $D \Downarrow U$ implies $\iota U = [\![D]\!]\rho_\iota$.*

Definition 2.7 (Labelled semantics). A *labelled configuration* is $(U, \mathcal{P})$ with $U \subseteq \mathbb{N}$ finite and $\mathcal{P}$ a finite multiset of closed processes whose names are free names and symbols ν_k. The τ-steps $(U, \mathcal{P}) \xrightarrow{\tau} (U, \mathcal{P}')$ are the steps (Red Nil), (Red Par), (Red Repl), (Red Eval 1), (Red Eval 2), (Red Cond 1′), (Red Cond 2′) and (Red I/O′) between two members of $\mathcal{P}$, with $=_E$ read on $\mathcal{M}$. The visible steps are

$$(U, \mathcal{P} \cup \{\mathsf{out}(N, M); P\}) \xrightarrow{\mathsf{out}([N],[M])} (U, \mathcal{P} \cup \{P\}),$$
$$(U, \mathcal{P} \cup \{\mathsf{in}(N, x); P\}) \xrightarrow{\mathsf{in}([N],[M])} (U, \mathcal{P} \cup \{P\{M/x\}\}) \quad \text{for every closed term } M,$$
$$(U, \mathcal{P} \cup \{\mathsf{new}\ a; P\}) \xrightarrow{\mathsf{new}(k)} (U \cup \{k\}, \mathcal{P} \cup \{P\{\nu_k/a\}\}) \quad \text{for every } k \notin U,$$
$$(U, \mathcal{P} \cup \{\mathsf{event}(M); P\}) \xrightarrow{\mathsf{ev}([M])} (U, \mathcal{P} \cup \{P\}).$$

With $\overset{\alpha}{\Longrightarrow} = \overset{\tau}{\to}{}^* \overset{\alpha}{\to} \overset{\tau}{\to}{}^*$, the set of *weak traces* is

$$\mathrm{Tr}(U, \mathcal{P}) = \{\alpha_1 \cdots \alpha_n \in \mathrm{Act}^* \mid (U, \mathcal{P}) \overset{\alpha_1}{\Longrightarrow} \cdots \overset{\alpha_n}{\Longrightarrow}\}.$$

Theorem 2.8 (Honest processes). *For every closed process P whose names are free names, $[\![P]\!]\rho_0 = \mathrm{Tr}(\emptyset, \{P\})$. For all finite multisets $\mathcal{P}, \mathcal{Q}$ of such processes, $\mathrm{Tr}(\emptyset, \mathcal{P} \uplus \mathcal{Q}) = \mathrm{Tr}(\emptyset, \mathcal{P}) \mid \mathrm{Tr}(\emptyset, \mathcal{Q})$.*

Definition 2.9 (Observation). For a trace Tr from $\mathcal{C}_Q$, $\mathrm{obs}(\mathrm{Tr}) \in \mathrm{Act}^*$ is the word that records $\mathsf{new}(\iota a')$ for each step (Red Res) creating a' and $\mathsf{ev}(\iota M)$ for each step (Red Event) on $\mathsf{event}(M)$, in the order of the steps, and nothing for the other steps. For $t \in \mathrm{Act}^*$, $t{\restriction_{\mathsf{new,ev}}}$ is the subword of new and ev actions.

Let $\mathcal{A}_0 = \mathrm{Attacks}([\![P_0]\!]\rho_0)$.

Theorem 2.10 (Attacks). *The following hold.*

(S) For every $\mathcal{N}_{\mathrm{pub}}$-adversary Q and every trace Tr *from $\mathcal{C}_Q$ there is $t \in \mathcal{A}_0$ with $t{\restriction_{\mathsf{new,ev}}} = \mathrm{obs}(\mathrm{Tr})$ such that $t\,\mathsf{say}(\iota M) \in \mathcal{A}_0$ for every M for which* Tr *satisfies* $\mathsf{attacker}(M)$.

(C) For every $t \in \mathcal{A}_0$ there are an $\mathcal{N}_{\mathrm{pub}}$-adversary Q, a trace Tr *from $\mathcal{C}_Q$ and an injection ι with $\mathrm{obs}(\mathrm{Tr}) = t{\restriction_{\mathsf{new,ev}}}$ such that, if $t = t'\,\mathsf{say}(m)$, then* Tr *satisfies* $\mathsf{attacker}(M)$ *for some closed term M with $\iota M = m$.*

Theorem 2.11 (Secrecy). *Let M be a closed term with $\mathrm{fn}(M) \subseteq \mathcal{N}_{\mathrm{pub}} \cup \mathcal{N}_{\mathrm{priv}}$ and let $s = \rho_0 M$. Then $\mathrm{Secret}([\![P_0]\!]\rho_0, s)$ holds if and only if no trace from $\mathcal{C}_Q$, for any $\mathcal{N}_{\mathrm{pub}}$-adversary Q, satisfies $\mathsf{attacker}(M)$. If $\mathcal{N}_{\mathrm{pub}} \neq \emptyset$, this is the condition that P_0 preserves the secrecy of M from $\mathcal{N}_{\mathrm{pub}}$, by Lemma 2.4.*

Remark 2.12. When $\mathcal{N}_{\mathrm{pub}} = \emptyset$, P_0 preserves the secrecy of every M in the sense of Definition 2.1(3), whereas $\mathrm{Secret}([\![P_0]\!]\rho_0, \rho_0 M)$ fails whenever $\emptyset \vdash \rho_0 M$, for instance for a public constant M.

Theorem 2.13 (Correspondences)**.** *Let $F, H_1, \ldots, H_m$ be facts over X such that every H_j is an event fact. Then P_0 satisfies $F \rightsquigarrow \bigvee_{j=1}^{m} H_j$ against $\mathcal{N}_{\mathrm{pub}}$-adversaries if and only if* $\mathrm{Corr}([\![P_0]\!]\rho_0, \mathrm{Hyp}, \mathrm{Concl})$ *holds for the relations of Definition 1.21.*

Remark 2.14. The restriction on the H_j cannot be dropped. For $H_j = \mathsf{attacker}(M)$, Concl demands an action $\mathsf{say}(\sigma M)$, which Enemy is never obliged to perform: with $\mathcal{N}_{\mathrm{pub}} = \{c\}$ and $P_0 = \mathsf{out}(c, s); \mathsf{event}(e)$, P_0 satisfies $\mathsf{event}(e) \rightsquigarrow \mathsf{attacker}(s)$, while the one-action trace $\mathsf{ev}(e)$ lies in $\mathcal{A}_0$ and no say precedes $\mathsf{ev}(e)$ in it, so Corr fails. Conclusions $\bigwedge_k H_{jk}$ with $l_j > 1$ have no reading in Definition 1.21, since Corr admits a single witness b.